

## Design Project I: Lab 2 Chipchat

### Example 1: (Final Status: Successful)

#### Prompt:

I am trying to create a Verilog model with name `binary_to_bcd_converter` for a binary to binary-coded-decimal converter. It must meet the following specifications:

- Inputs:
  - Binary input (5-bits, signal name: `binary_input`)
- Outputs:
  - BCD (8-bits: 4-bits for the 10's place and 4-bits for the 1's place, signal name: `bcd_output`)

How would I write a design that meets these specifications?

#### Output:

```
module binary_to_bcd_converter (
    // Inputs
    input      [4:0] binary_input,

    // Outputs
    output reg [7:0] bcd_output
);

    // Internal registers to hold the individual BCD digits for clarity.
    // A 5-bit input (max 31) needs a tens digit up to 3 and a ones digit up to 9.
    // 4 bits are sufficient for each.
    reg [3:0] tens_digit;
    reg [3:0] ones_digit;

    // This block describes combinational logic. It will re-evaluate
    // whenever the `binary_input` signal changes.
    always @(*) begin
        // The division operator gives the integer part of the result.
        // For example, 27 / 10 = 2.
        tens_digit = binary_input / 10;

        // The modulus operator gives the remainder of the division.
        // For example, 27 % 10 = 7.
        ones_digit = binary_input % 10;

        // Concatenate the two 4-bit digits to form the final 8-bit BCD output.
        // The tens digit is the most significant nibble.
    end
endmodule
```

```

        bcd_output = {tens_digit, ones_digit};
    end

endmodule

```

Analysis: The initial prompt given in the Chipchat Python notebook was able to successfully generate a Verilog module that passed the provided testbench.

## Example 2: Sequence Detector (Final Status: Successful)

### Initial Prompt:

I am trying to create a Verilog model for a sequence detector. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset
  - Data (3 bits)
- Outputs:
  - Sequence found

While enabled, it should detect the following sequence of binary input values:

- 0b001
- 0b101
- 0b110
- 0b000
- 0b110
- 0b110
- 0b011
- 0b101

How would I write a design that meets these specifications?

### Initial Output:

```

module sequence_detector (
    input  wire      clk,           // Clock input
    input  wire      rst_n,        // Active-low reset
    input  wire [2:0] data,         // 3-bit data input
    output reg       seq_found     // Output indicating sequence found
);

// State encoding
typedef enum reg [3:0] {

```

```

    S0, // Initial state
    S1, // Detected 0b001
    S2, // Detected 0b101
    S3, // Detected 0b110
    S4, // Detected 0b000
    S5, // Detected 0b110 again
    S6, // Detected 0b011
    S7  // Detected 0b101 again
} state_t;

state_t current_state, next_state;

// Sequential logic: State transition on clock
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        current_state <= S0; // Reset to initial state
    else
        current_state <= next_state; // Transition to next state
end

// Next state logic
always @* begin
    // Default values
    next_state = current_state;
    seq_found = 1'b0; // By default, no sequence found

    case (current_state)
        S0: if (data == 3'b001) next_state = S1;
            else next_state = S0;
        S1: if (data == 3'b101) next_state = S2;
            else next_state = S0;
        S2: if (data == 3'b110) next_state = S3;
            else next_state = S0;
        S3: if (data == 3'b000) next_state = S4;
            else next_state = S0;
        S4: if (data == 3'b110) next_state = S5;
            else next_state = S0;
        S5: if (data == 3'b110) next_state = S5; // Remain in state S5
            else if (data == 3'b011) next_state = S6;
            else next_state = S0;
        S6: if (data == 3'b101) begin
                next_state = S7;
            end
    endcase
end

```

```

        seq_found = 1'b1;    // Sequence found
    end else next_state = S0;
S7: if (data == 3'b101) begin
        seq_found = 1'b1;    // Sequence found again
        next_state = S7;     // Remain in state S7
    end else next_state = S0;
    default: next_state = S0; // Default case to reset
endcase
end
endmodule

```

Analysis: needed to just revise the input names such that it matched the testbench inputs and outputs

### Second Prompt:

I am trying to create a Verilog model for a sequence detector. It must meet the following specifications:

- Inputs:
  - Clock, call it shorthand clk
  - Active-low reset, make sure the name of this is reset\_n
  - Data (3 bits)
- Outputs:
  - Sequence found, make sure the wire is named sequence\_found

While enabled, it should detect the following sequence of binary input values:

- 0b001
- 0b101
- 0b110
- 0b000
- 0b110
- 0b110
- 0b011
- 0b101

How would I write a design that meets these specifications?

### Second Output:

```

module sequence_detector (
    input wire clk,
    input wire reset_n,
    input wire [2:0] data,

```

```

    output reg sequence_found
);

// State Encoding
typedef enum reg [3:0] {
    S0 = 4'b0000, // Initial State
    S1 = 4'b0001, // Detected 0b001
    S2 = 4'b0010, // Detected 0b101
    S3 = 4'b0011, // Detected 0b110
    S4 = 4'b0100, // Detected 0b000
    S5 = 4'b0101, // Detected 0b110 (again)
    S6 = 4'b0110, // Detected 0b110 (third time)
    S7 = 4'b0111 // Detected 0b011
} state_t;

state_t current_state, next_state;

// State Transition Logic
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        current_state <= S0; // Reset to initial state
        sequence_found <= 0; // Clear sequence_found on reset
    end else begin
        current_state <= next_state; // Update to the next state
    end
end

// Next State Logic based on current state and input data
always @(*) begin
    // Default assignment for next state
    next_state = current_state;
    sequence_found = 0; // Default output

    case (current_state)
        S0: begin
            if (data == 3'b001) next_state = S1;
        end
        S1: begin
            if (data == 3'b101) next_state = S2;
        end
        S2: begin
            if (data == 3'b110) next_state = S3;
        end
    endcase
end

```

```

        end
        S3: begin
            if (data == 3'b000) next_state = S4;
        end
        S4: begin
            if (data == 3'b110) next_state = S5; // Detected 0b110 again
        end
        S5: begin
            if (data == 3'b110) next_state = S6; // Detected 0b110 again
        end
        S6: begin
            if (data == 3'b011) next_state = S7; // Detected 0b011
        end
        S7: begin
            if (data == 3'b101) begin
                next_state = S2; // Cycle back to detect 0b101 again after 0b011
                sequence_found = 1; // We found the sequence
            end
        end
    endcase
end
endmodule

```

Analysis: Passed the provided testbench

### Example 3: Shift Register (Status: Partially Successful)

Prompt: Decided to just label the wire and also make sure that the initial data out was initialized

I am trying to create a Verilog model for a shift register. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset
  - Data (1 bit), call it data\_in
  - Shift enable, call this shift\_enable
- Outputs:
  - Data (8 bits), call this data\_out

Make sure to set the initial data out

How would I write a design that meets these specifications?

Output:

```
module shift_register (
    input wire clk,           // Clock input
    input wire reset_n,       // Active-low reset
    input wire data_in,       // 1-bit data input
    input wire shift_enable,   // Shift enable input
    output reg [7:0] data_out  // 8-bit data output
);

// Initial value for data_out
initial begin
    data_out = 8'b00000000;    // Initialize data_out to 0
end

// Always block triggered on the rising edge of the clock or on reset
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        // Active-low reset (reset the shift register)
        data_out <= 8'b00000000; // Reset data_out to 0
    end else if (shift_enable) begin
        // Shift operation
        data_out <= {data_out[6:0], data_in}; // Shift left and input data_in at
LSB
    end
end

endmodule
```

Analysis: Didn't fully pass the testbench, but I think there are issues with the testbench. The 64-bit number for the test\_case\_data\_out should be going from [0:63] to match the pattern, Test case 4 is flawed since you need some form of time delay in order for the output signals to propagate. Test case 6 also doesn't work since it should reset but the expected output is not a reset.

Example 4: Abro State Machine (Status: Partially Successful)

Initial Prompt:

I am trying to create a Verilog model for an ABRO state machine. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset, use rst\_n for the input name
  - A
  - B
- Outputs:
  - O
  - State, use the capitalized version for the wire name

Other than the main output from ABRO machine, it should output the current state of the machine for use in verification.

Call the module abro\_state\_machine

The states for this state machine should be one-hot encoded.

How would I write a design that meets these specifications?

### Initial Output:

```
module abro_state_machine (  
    input wire clk,  
    input wire rst_n,  
    input wire A,  
    input wire B,  
    output reg O,  
    output reg [3:0] State  
);  
    // Define state parameters using localparams  
    localparam IDLE      = 4'b0001;  
    localparam STATE_A   = 4'b0010;  
    localparam STATE_B   = 4'b0100;  
    localparam OUTPUT    = 4'b1000;  
  
    // State variable  
    reg [3:0] current_state, next_state;  
  
    // State transition on rising edge of clock  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            current_state <= IDLE; // Asynchronous reset to IDLE  
        end else begin  
            current_state <= next_state; // Update current state
```

```

    end

end

// Logic for next state and output generation
always @(*) begin
    // Default assignments
    next_state = current_state; // stay in current state by default
    O = 1'b0; // default output
    State = current_state; // output current state

    case (current_state)
        IDLE: begin
            if (A) begin
                next_state = STATE_A; // Transition to STATE_A if A is high
            end else if (B) begin
                next_state = STATE_B; // Transition to STATE_B if B is high
            end
        end

        STATE_A: begin
            if (B) begin
                next_state = OUTPUT; // Transition to OUTPUT if B is high
            end else begin
                next_state = IDLE; // Return to IDLE if not
            end
        end

        STATE_B: begin
            if (A) begin
                next_state = OUTPUT; // Transition to OUTPUT if A is high
            end else begin
                next_state = IDLE; // Return to IDLE if not
            end
        end

        OUTPUT: begin
            O = 1'b1; // Output high during OUTPUT state
            if (!A && !B) begin
                next_state = IDLE; // Return to IDLE if both inputs are low
            end
        end
    endcase
end

```

```
end  
endmodule
```

Highlighted example of how I needed to modify this prompt. I needed to specify specifically the behavior of the ABRO machine based on the testbench since I the model had a different idea of what an ABRO machine was supposed to do.

A lot of different prompts later: This was the prompt that produced the best results, different specifications would tend to either fail on earlier test cases, but this one at least made it through the initial 4

```
I am trying to create a Verilog model for an ABRO state machine. It must meet the following specifications:
```

- Inputs:
  - Clock
  - Active-low reset, use rst\_n for the input name
  - A
  - B
- Outputs:
  - O
  - State, use the capitalized version for the wire name

```
Other than the main output from ABRO machine, it should output the current state of the machine for use in verification.
```

```
Call the module abro_state_machine
```

```
Do not use 4'b0000 as a state
```

```
This ABRO machine should see A high first, then B, meaning that if B was seen high first, it does not mean a subsequent A will produce a 1 output.
```

```
Use 4'b0001, 4'b0010, and 4'b0100 as the state values, essentially one-hot encoded
```

```
How would I write a design that meets these specifications?
```

### Best Output Produced:

```
module abro_state_machine (  
    input wire clk,  
    input wire rst_n,  
    input wire A,  
    input wire B,  
    output reg O,
```

```

    output reg [3:0] State
);

// Define state encoding
parameter STATE_A = 4'b0001; // A detected
parameter STATE_B = 4'b0010; // B detected
parameter STATE_AB = 4'b0100; // A followed by B

// Current state and next state variables
reg [3:0] current_state, next_state;

// Synchronous logic for state transition
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        current_state <= STATE_A; // Reset to initial state
        O <= 0; // Reset output
    end else begin
        current_state <= next_state; // Update state
    end
end

// Combinational logic for next state and output
always @(*) begin
    // Default values for next_state and output
    next_state = current_state;
    O = 0;

    case (current_state)
        STATE_A: begin
            if (A) begin
                next_state = STATE_A; // Stay in state A
            end else if (B) begin
                next_state = STATE_B; // Transition to state B
            end
        end

        STATE_B: begin
            if (A) begin
                next_state = STATE_AB; // Transition to state AB
            end else begin
                next_state = STATE_B; // Stay in state B
            end
        end
    endcase
end

```

```

        end

        STATE_AB: begin
            O = 1; // Output is high when in state AB
            // Optionally, define your transition behavior here
            // For example, you could return to initial state or hold.
            if (!A && !B) begin
                next_state = STATE_A; // Return to state A on both low inputs
            end else if (A && !B) begin
                next_state = STATE_A; // Stay in state A
            end else if (!A && B) begin
                next_state = STATE_B; // Stay in state B
            end
        end
    end

    default: begin
        next_state = STATE_A; // Safe default
    end
endcase
end

// Assign current state to output State
always @(*) begin
    State = current_state; // Output the current state
end

endmodule

```

Failed on test case 5, since the output was still kept high, only problem is when trying to do additional prompts that fixed it, it would mess with earlier test cases and could never explicitly get the LLM to fix it.

### Example 5: Dice Roller (Status: Successful)

#### Prompt:

I am trying to create a Verilog model for a simulated dice roller. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset
  - Die select (2-bits)
  - Roll
- Outputs:

- Rolled number (up to 8-bits)

The design should simulate rolling either a 4-sided, 6-sided, 8-sided, or 20-sided die, based on the input die select. It should roll when the roll input goes high and output the random number based on the number of sides of the selected die.

How would I write a design that meets these specifications?

Output:

```
module dice_roller (
    input wire clk,                // Clock input
    input wire rst_n,              // Active-low reset
    input wire [1:0] die_select,    // 2-bits for die selection
    input wire roll,                // Roll trigger
    output reg [7:0] rolled_number  // Output rolled number (8-bits)
);

    // Define the die sizes based on the die_select
    parameter DIE_4_SIDES = 2'b00;
    parameter DIE_6_SIDES = 2'b01;
    parameter DIE_8_SIDES = 2'b10;
    parameter DIE_20_SIDES = 2'b11;

    reg [7:0] max_number; // Max number based on die selected
    reg rolling;          // Flag to indicate rolling in progress

    // Random number generation (simple Linear Congruential Generator for simulation)
    reg [7:0] rand_num; // Store random number

    // Simple LFSR or counter to generate pseudo-random numbers
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rand_num <= 8'h1; // Seed value
        end else begin
            // Update random number (example LCG, multiply by a constant and mod)
            rand_num <= (rand_num * 1103515245 + 12345) % 256;
        end
    end

    // Determine max number based on die_select
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
```

```

        max_number <= 0;
    end else begin
        case (die_select)
            DIE_4_SIDES: max_number <= 4;
            DIE_6_SIDES: max_number <= 6;
            DIE_8_SIDES: max_number <= 8;
            DIE_20_SIDES: max_number <= 20;
            default: max_number <= 0; // In case of undefined input
        endcase
    end
end

// Handle roll input
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rolled_number <= 0;
        rolling <= 0;
    end else begin
        if (roll && !rolling) begin
            rolling <= 1; // Start rolling
            rolled_number <= 1 + (rand_num % max_number); // Generate rolled number
        end else if (!roll) begin
            rolling <= 0; // Stop rolling when roll is low
        end
    end
end

end
endmodule

```

Analysis: Worked first try with the already provided prompt in notebook

## Example 6: LFSR (Status: Successful)

### Prompt:

I am trying to create a Verilog model for an LFSR. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset
- Outputs:
  - Data (8-bits)

The initial state should be 10001010, and the taps should be at locations 1, 4, 6, and 7.

How would I write a design that meets these specifications?

Output:

```
module lfsr (
    input wire    clk,
    input wire    reset_n,
    output wire [7:0] data
);

    reg [7:0] lfsr_reg;

    // Feedback: XOR of bits at positions 0, 3, 5, and 6 (taps 1, 4, 6, 7)
    wire feedback;
    assign feedback = lfsr_reg[0] ^ lfsr_reg[3] ^ lfsr_reg[5] ^ lfsr_reg[6];

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            lfsr_reg <= 8'b10001010; // Initial state
        end else begin
            lfsr_reg <= {lfsr_reg[6:0], feedback}; // Shift left, new bit enters at
LSB
        end
    end

    assign data = lfsr_reg;
endmodule
```

Analysis: Worked with initial prompt

Example 7: Sequence Generator (Status: Successful)

Prompt:

I am trying to create a Verilog model for a sequence generator. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset

- Enable
- Outputs:
  - Data (8 bits)

While enabled, it should generate an output sequence of the following hexadecimal values and then repeat:

- 0xAF
- 0xBC
- 0xE2
- 0x78
- 0xFF
- 0xE2
- 0x0B
- 0x8D

How would I write a design that meets these specifications?

**Output:**

```
module sequence_generator (
    input wire clk,
    input wire reset_n, // Active-low reset
    input wire enable,
    output reg [7:0] data
);

// State encoding
localparam [2:0]
    STATE_0 = 3'b000,
    STATE_1 = 3'b001,
    STATE_2 = 3'b010,
    STATE_3 = 3'b011,
    STATE_4 = 3'b100,
    STATE_5 = 3'b101,
    STATE_6 = 3'b110,
    STATE_7 = 3'b111;

// Current state and next state variables
reg [2:0] current_state, next_state;

// Define the sequence of values
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
```

```

        current_state <= STATE_0; // Reset to the initial state
    end else if (enable) begin
        current_state <= next_state; // Update state when enabled
    end
end

// Output logic and state transition logic
always @(*) begin
    case (current_state)
        STATE_0: begin
            data = 8'hAF; // 0xAF
            next_state = STATE_1;
        end
        STATE_1: begin
            data = 8'hBC; // 0xBC
            next_state = STATE_2;
        end
        STATE_2: begin
            data = 8'hE2; // 0xE2
            next_state = STATE_3;
        end
        STATE_3: begin
            data = 8'h78; // 0x78
            next_state = STATE_4;
        end
        STATE_4: begin
            data = 8'hFF; // 0xFF
            next_state = STATE_5;
        end
        STATE_5: begin
            data = 8'hE2; // 0xE2
            next_state = STATE_6;
        end
        STATE_6: begin
            data = 8'h0B; // 0x0B
            next_state = STATE_7;
        end
        STATE_7: begin
            data = 8'h8D; // 0x8D
            next_state = STATE_0; // Go back to the first state
        end
        default: begin

```

```

        data = 8'h00; // Default case (shouldn't occur)
        next_state = STATE_0; // Safety fallback
    end
endcase
end
endmodule

```

Analysis: Initial prompt successfully created Verilog module that passed testbench

## Example 8: Traffic Light (Status: Successful)

### Prompt:

I am trying to create a Verilog model for a traffic light state machine. It must meet the following specifications:

- Inputs:
  - Clock
  - Active-low reset
  - Enable
- Outputs:
  - Red
  - Yellow
  - Green

The state machine should reset to a red light, change from red to green after 32 clock cycles, change from green to yellow after 20 clock cycles, and then change from yellow to red after 7 clock cycles.

How would I write a design that meets these specifications?

### Output:

```

module traffic_light_fsm(
    input wire clk,           // Clock input
    input wire reset_n,       // Active-low reset
    input wire enable,        // Enable signal
    output reg red,           // Red light output
    output reg yellow,        // Yellow light output
    output reg green          // Green light output
);

    // State definitions
    localparam RED = 2'b00;

```

```

localparam GREEN = 2'b01;
localparam YELLOW = 2'b10;

reg [1:0] state;          // Current state
reg [4:0] counter;        // 5-bit counter to hold the number of clock cycles

// State transition logic
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        state <= RED;          // Reset to RED state
        counter <= 0;          // Reset counter
    end else if (enable) begin
        case (state)
            RED: begin
                if (counter == 31) begin // 32 clock cycles
                    state <= GREEN;
                    counter <= 0;        // Reset counter for the next state
                end else begin
                    counter <= counter + 1; // Increment counter
                end
            end
            GREEN: begin
                if (counter == 19) begin // 20 clock cycles
                    state <= YELLOW;
                    counter <= 0;        // Reset counter for the next state
                end else begin
                    counter <= counter + 1; // Increment counter
                end
            end
            YELLOW: begin
                if (counter == 6) begin // 7 clock cycles
                    state <= RED;
                    counter <= 0;        // Reset counter for the next state
                end else begin
                    counter <= counter + 1; // Increment counter
                end
            end
            default: begin

```

```

        state <= RED;           // Default to RED in case of an unknown
state
        counter <= 0;           // Reset counter
    end
endcase
end
end

// Output logic based on the current state
always @(*) begin
    // Default output states to low
    red = 1'b0;
    yellow = 1'b0;
    green = 1'b0;

    case (state)
        RED:    red = 1'b1;
        GREEN:  green = 1'b1;
        YELLOW: yellow = 1'b1;
    endcase
end
endmodule

```

Analysis: Was able to one-shot this example as well

